IT342 – System Integration and Architecture

MedBuddy

Project Development Phase 3

WEB Development

Prepared by	Aquino, Julianne Rose P.
Section	IT342-G2
Project Name	MedBuddy
Date	April 6, 2026
Phase	Phase 3 – Web Development
Status	Completed

1. Final Commit Information

Commit Message

```
IT342 Phase 3 - Web Main Feature Completed
```

Repository Details

Repository Name: IT342_MedBuddy_Aquino

Branch: main

Commit Hash: d74dc72e3dc6bb937a03bae650a86ad1c2bcfcf2

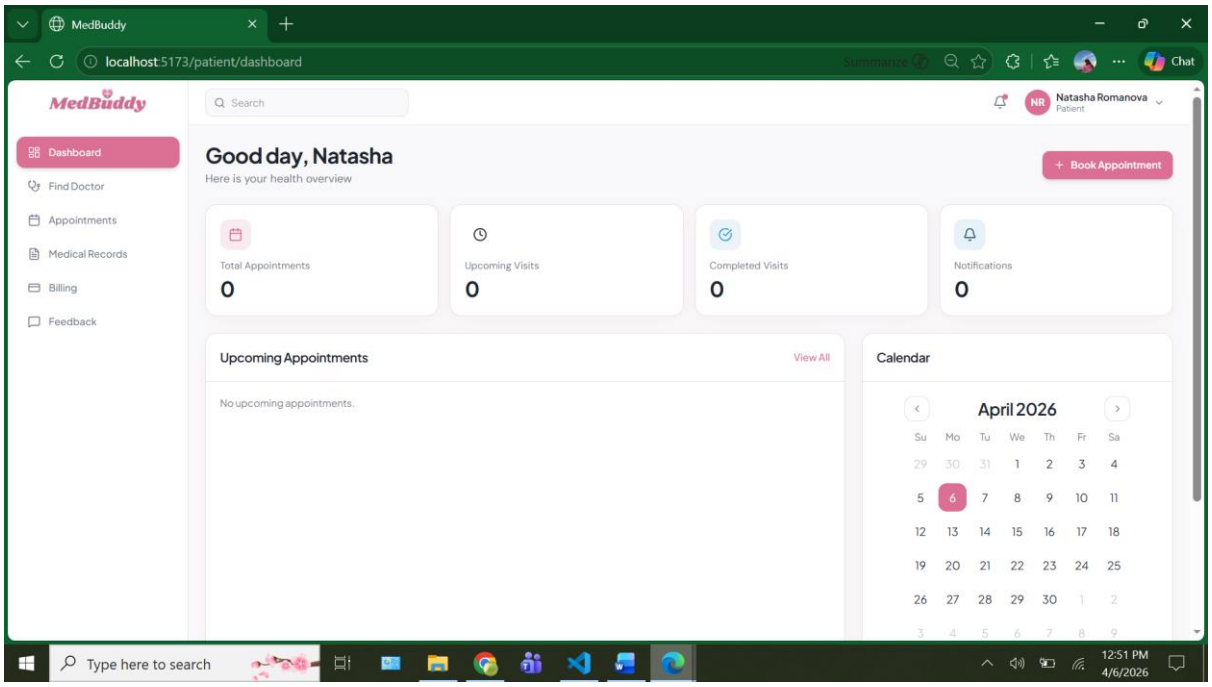
Commit Link: https://github.com/oblivjules/IT342_MedBuddy_Aquino/commit/d74dc72e3dc6bb937a03bae650a86ad1c2bcfcf2

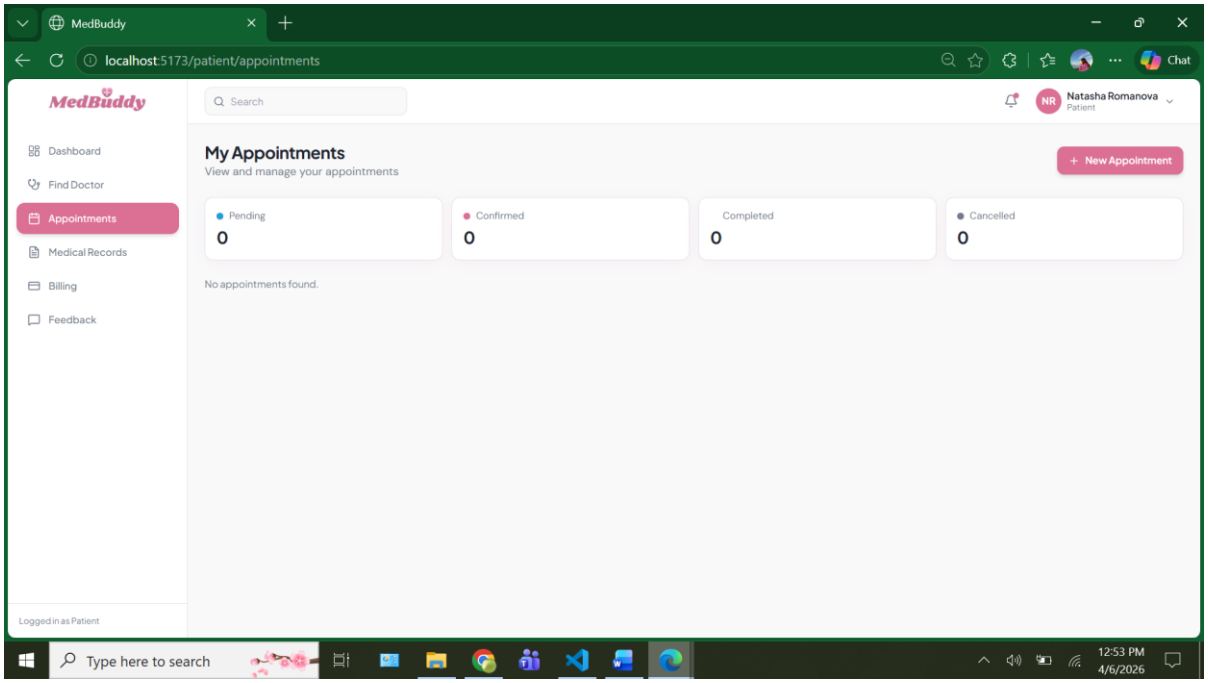
Repository Link: https://github.com/oblivjules/IT342_MedBuddy_Aquino.git

2. Screenshots of Implementation

Screenshot 1 – Dashboard and Appointments Page

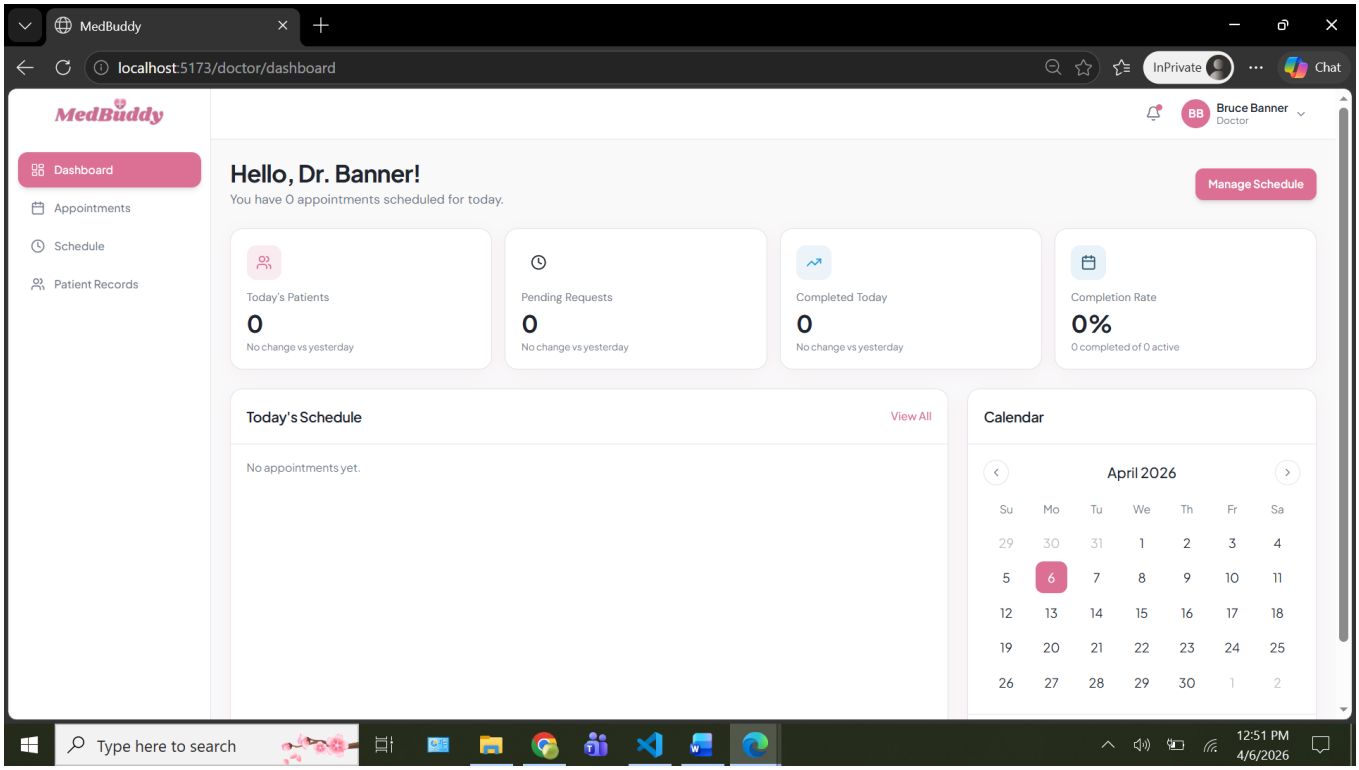
a. Patient Dashboard and Appointments Page

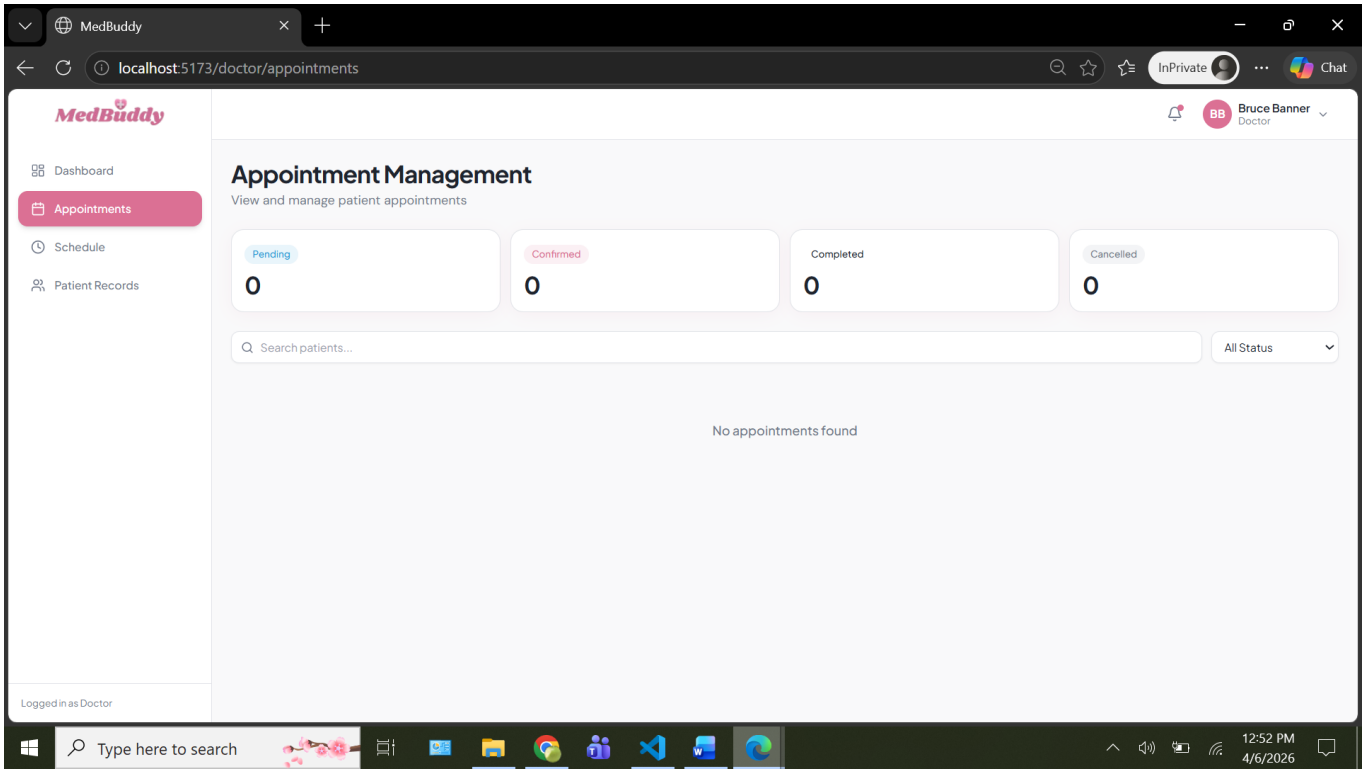




2. Screenshots of Implementation

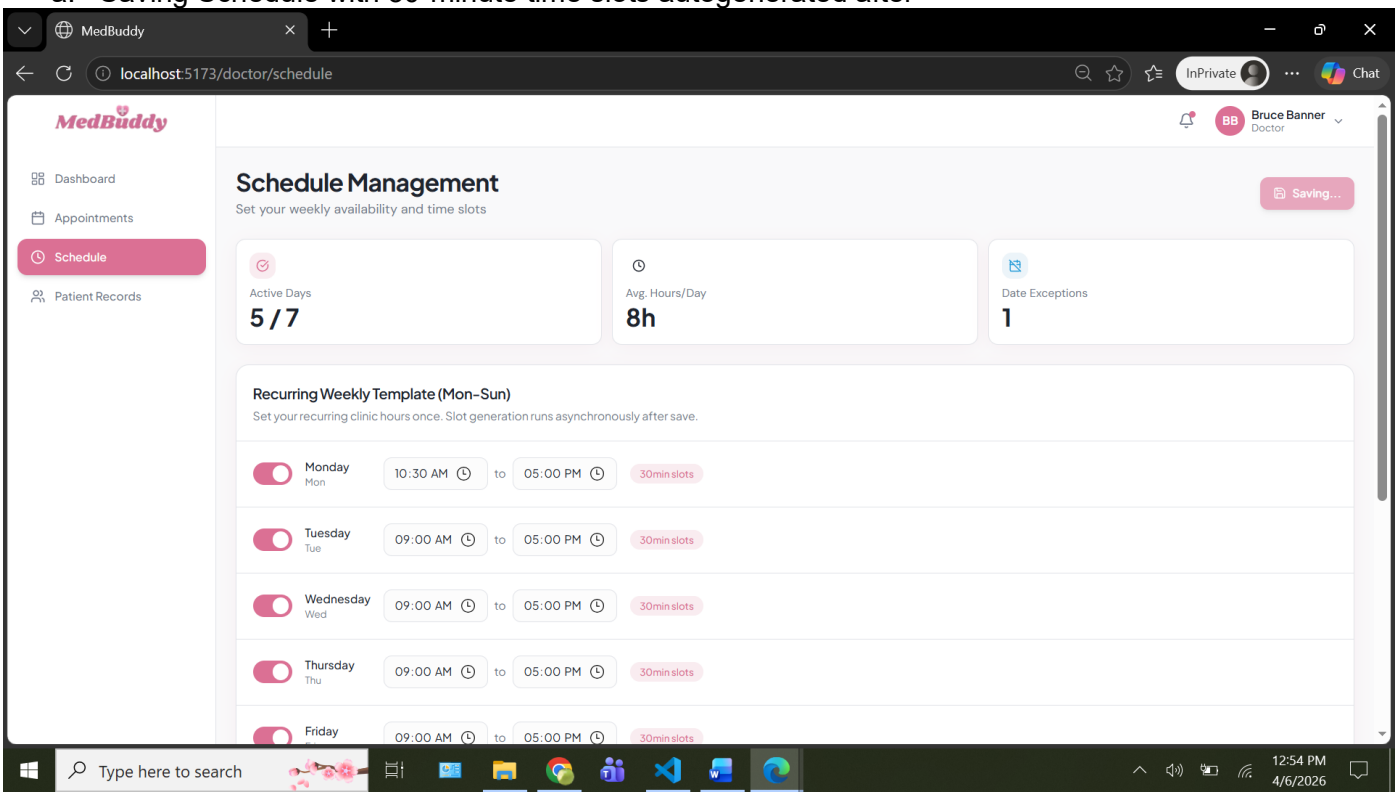
b. Doctor Dashboard and Appointments Page

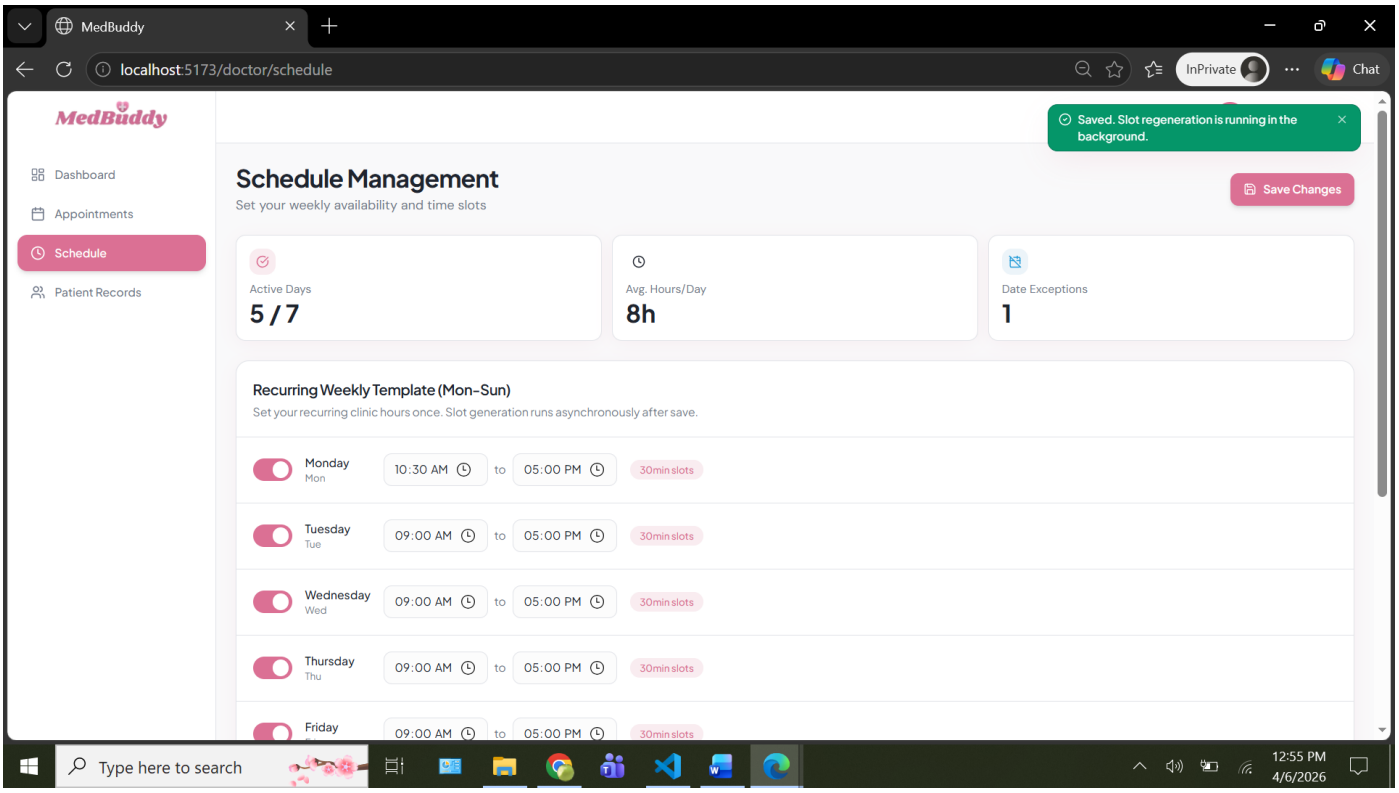




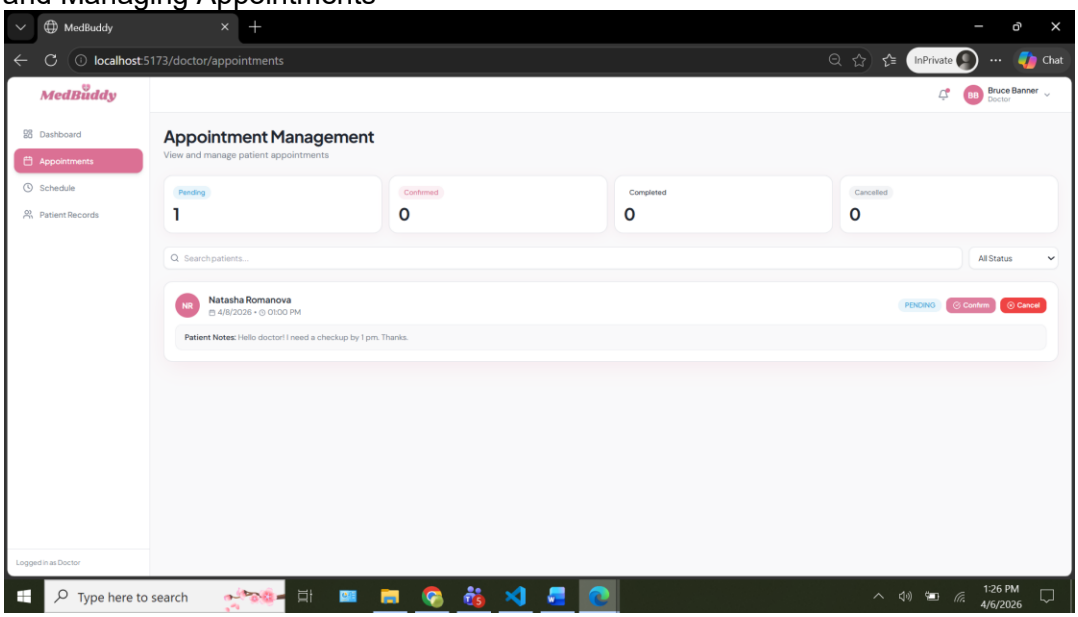
Screenshot 2 – Doctor Appointments Page and Managing Schedule Availability

a. Saving Schedule with 30-minute time slots autogenerated after

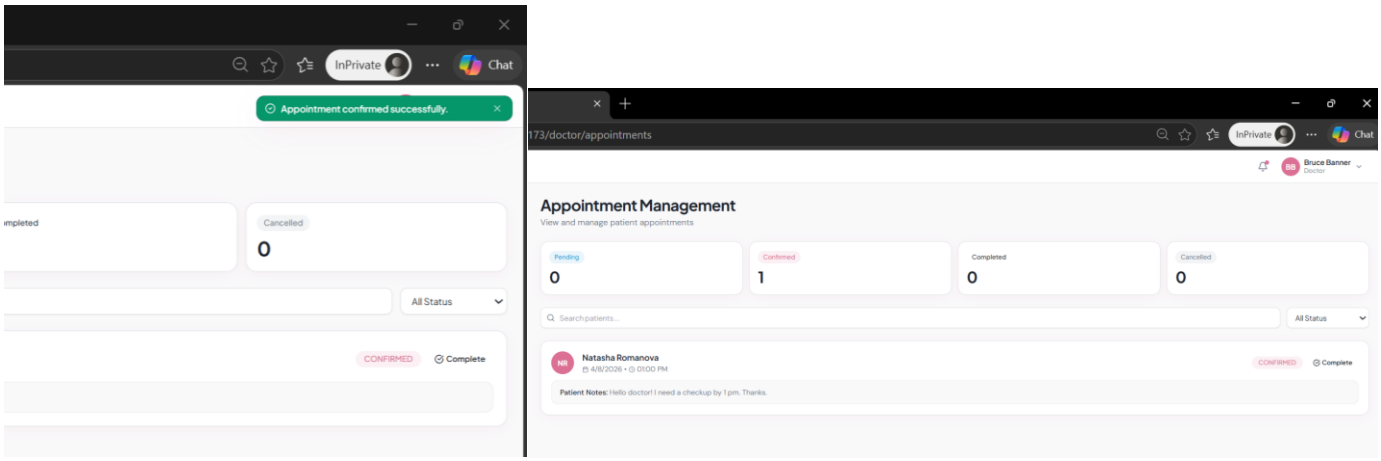




b. Viewing and Managing Appointments



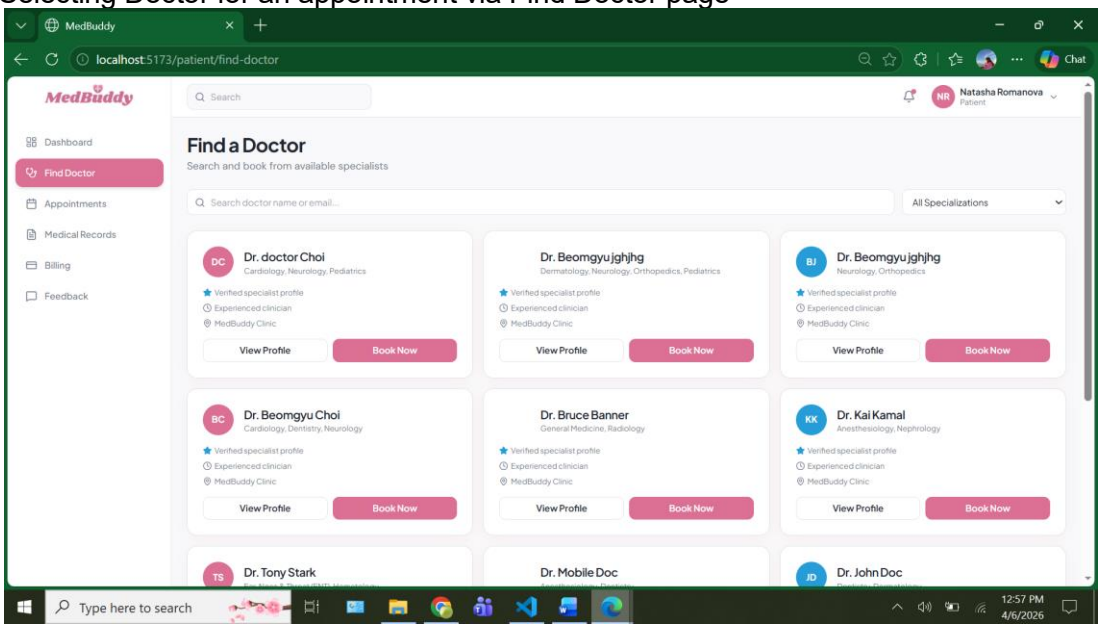
i. *Appointment booked by patient Natasha pending on doctor's end. (Check Screenshot 5)*



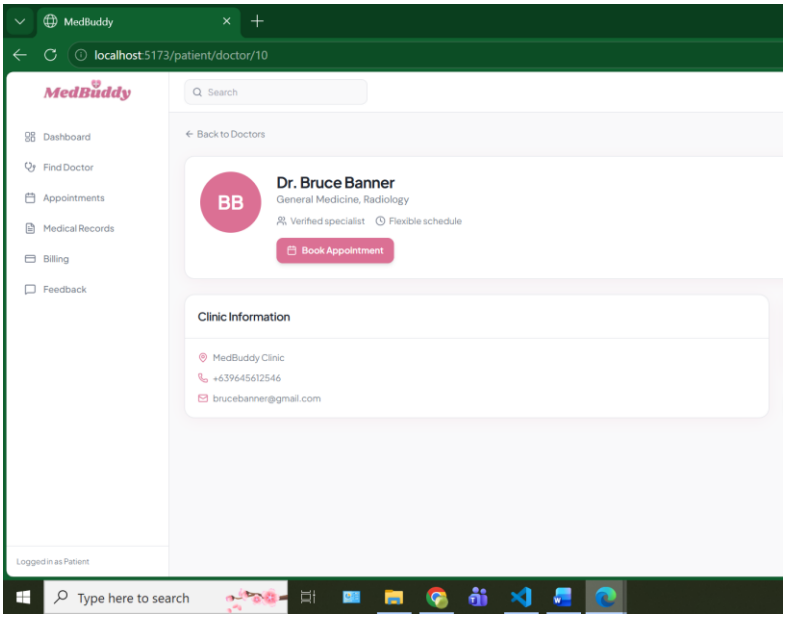
ii. *Doctor confirmed the appointment successfully.*

Screenshot 3 – Patient Appointment Page

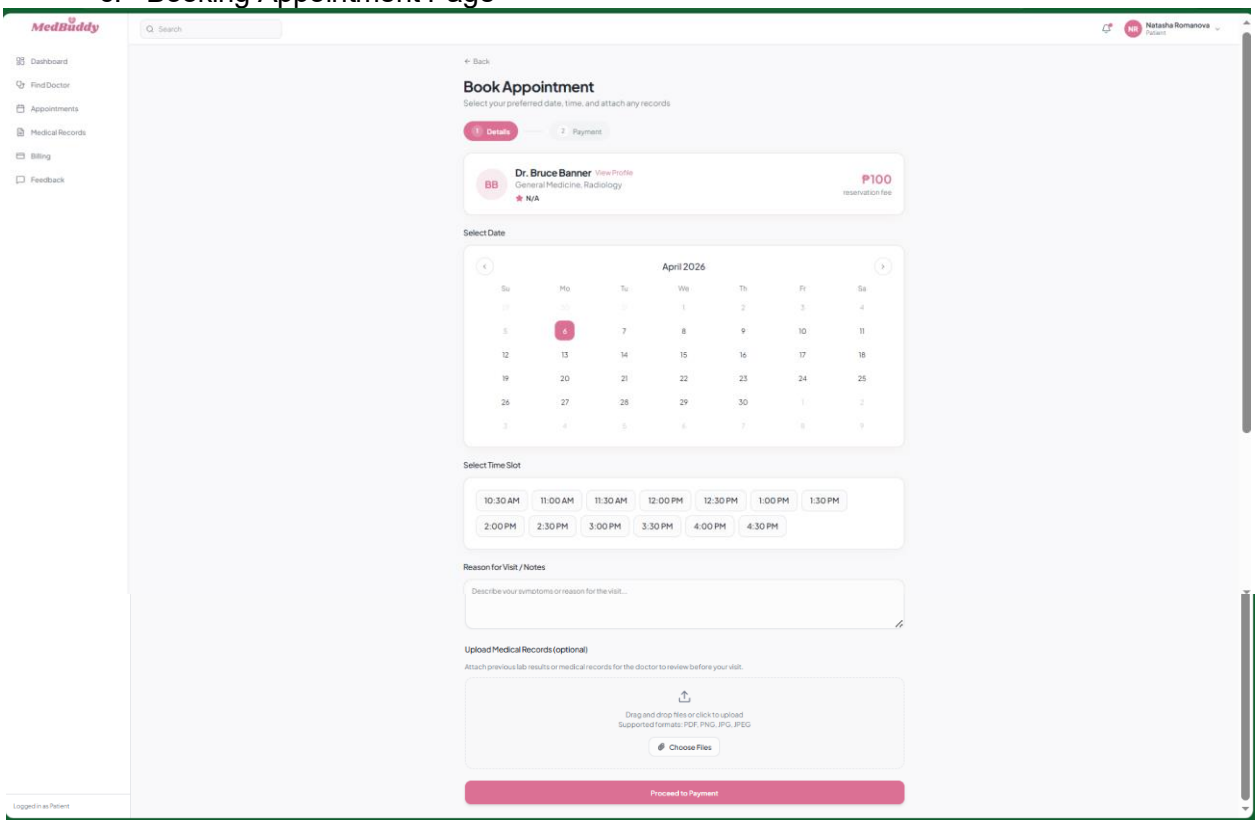
a. Selecting Doctor for an appointment via Find Doctor page



b. Viewing Doctor Profile

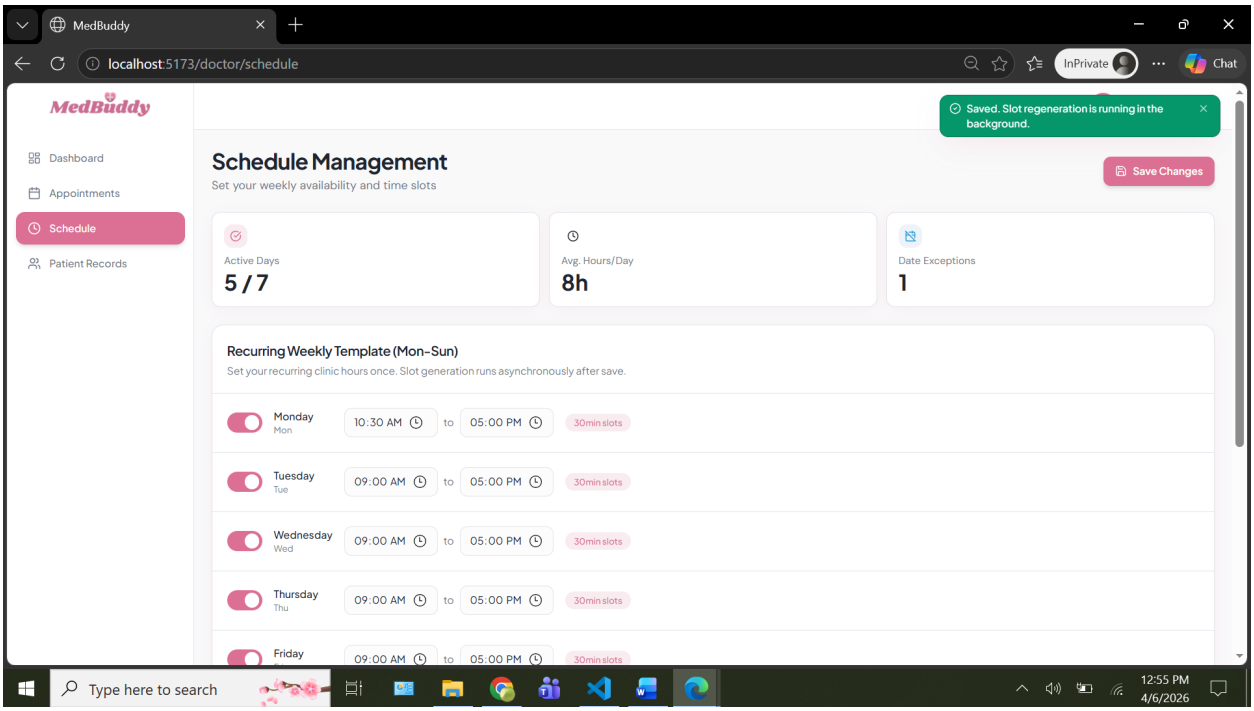


c. Booking Appointment Page



Screenshot 4 – Successful Saving of Doctor Availability/Schedule

a. Successful Save with Validation message



b. Database records for doctor schedule

available_date date	end_time time	start_time time	status varchar(20)	doctor_id bi
2026-04-09	00:30:00	00:00:00	UNAVAILABLE	10

iii. From doctor_availability table, which stores dates for exceptions (day-off, adjusted hours). In this case, April 9 is a holiday (Araw ng Kagitingan), so the doctor adds it as an "exception" date or day-off from regular schedule.

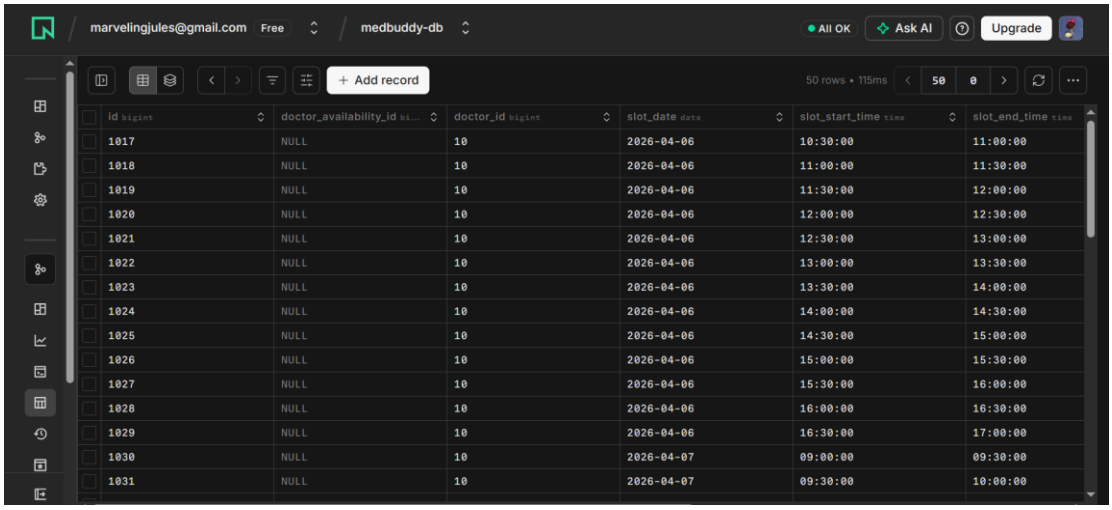
+ Add record

5 rows • 402ms 50 0

id bigint	doctor_id bigint	day_of_week integer	start_time time	end_time time	is_active boolean
16	10	0	10:30:00	17:00:00	TRUE
17	10	1	09:00:00	17:00:00	TRUE
18	10	2	09:00:00	17:00:00	TRUE
19	10	3	09:00:00	17:00:00	TRUE
20	10	4	09:00:00	17:00:00	TRUE

iv. From doctor_schedule_templates, which stores regular/available schedules of the doctor. In this case, the doctor's (Bruce Banner, doctor_id 10) regular schedule is from Monday to Friday, usually at 9:00 am to 5:00 pm, except for Monday, which I set to start at 10:30 am.

c. Database records for autogenerated 30-minute slots per appointment schedule

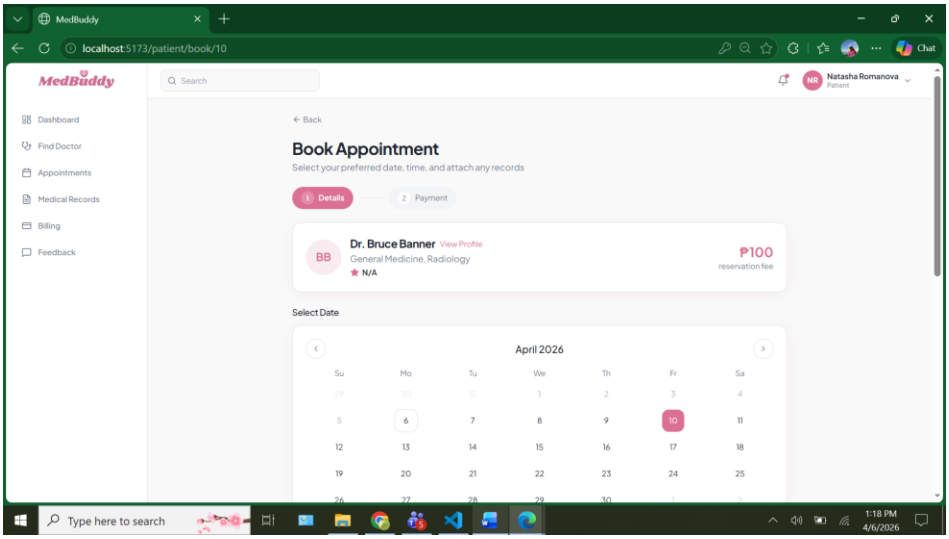


id bigint	doctor_availability_id bi...	doctor_id bigint	slot_date date	slot_start_time time	slot_end_time time
1017	NULL	10	2026-04-06	10:30:00	11:00:00
1018	NULL	10	2026-04-06	11:00:00	11:30:00
1019	NULL	10	2026-04-06	11:30:00	12:00:00
1020	NULL	10	2026-04-06	12:00:00	12:30:00
1021	NULL	10	2026-04-06	12:30:00	13:00:00
1022	NULL	10	2026-04-06	13:00:00	13:30:00
1023	NULL	10	2026-04-06	13:30:00	14:00:00
1024	NULL	10	2026-04-06	14:00:00	14:30:00
1025	NULL	10	2026-04-06	14:30:00	15:00:00
1026	NULL	10	2026-04-06	15:00:00	15:30:00
1027	NULL	10	2026-04-06	15:30:00	16:00:00
1028	NULL	10	2026-04-06	16:00:00	16:30:00
1029	NULL	10	2026-04-06	16:30:00	17:00:00
1030	NULL	10	2026-04-07	09:00:00	09:30:00
1031	NULL	10	2026-04-07	09:30:00	10:00:00

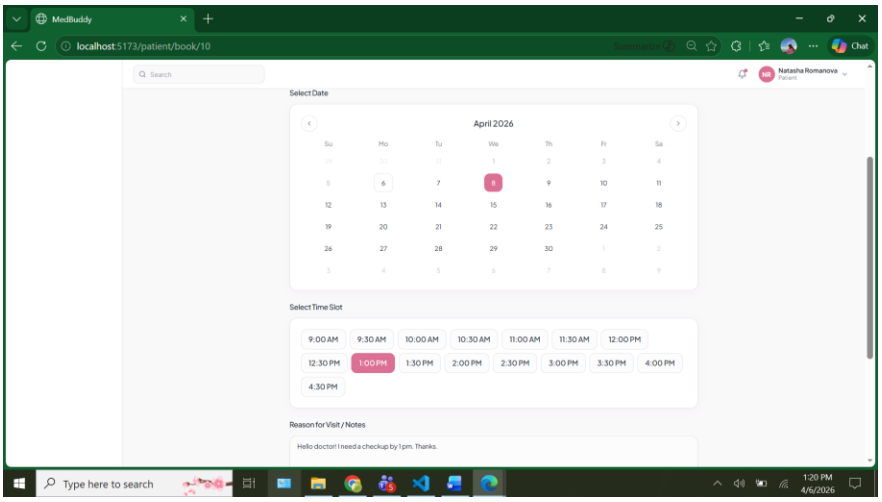
v. From appointment_slots table, which shows all the auto-generated 30-minute slots after schedule was saved. Added a constraint to exclude the last 30-minute slots of every schedule so that it will be allotted for a clinic/hospital's cutoff time.

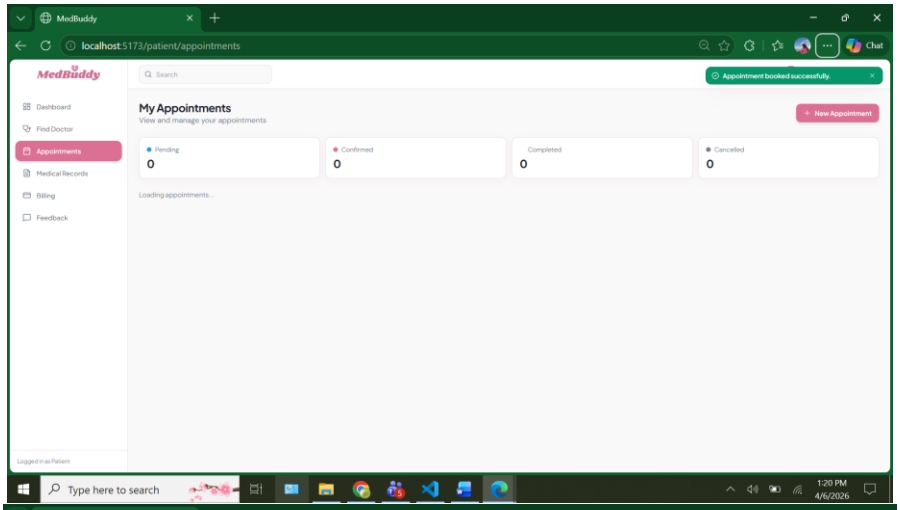
Screenshot 5 – Patient Successful Appointment Booking

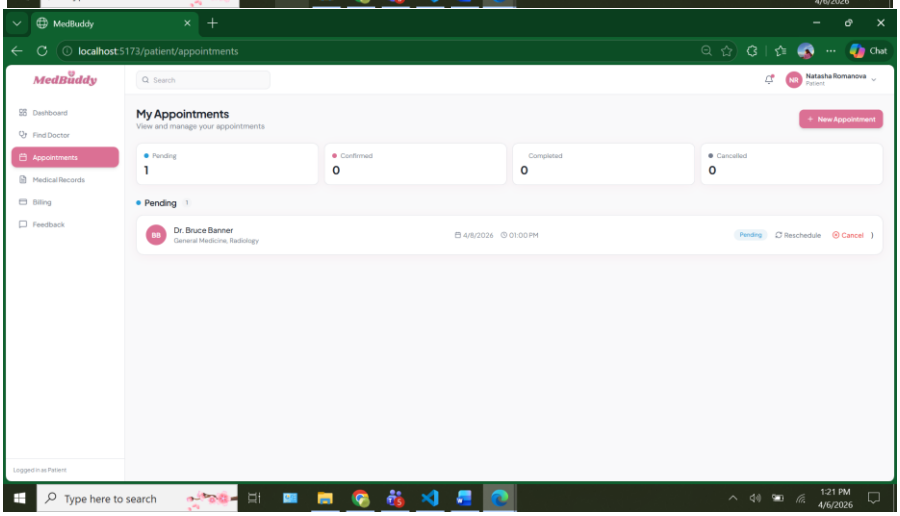
a. Successful Booking with Validation message



vi. Reservation fee was only added to the UI to follow the booking flow from the SDD. Payment module will be added later.

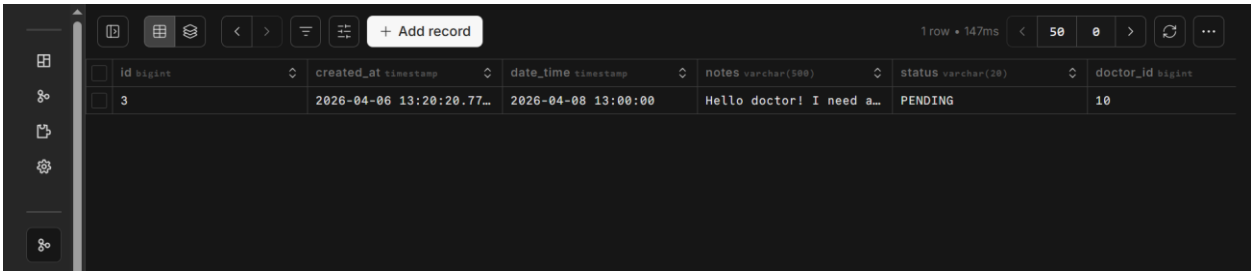




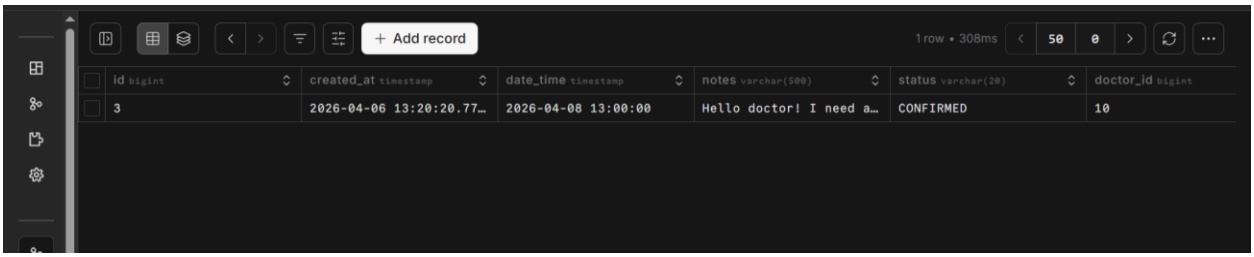


vii. Booked appointment now shows as “pending”.

b. Database record for appointment



id bigint	created_at timestamp	date_time timestamp	notes varchar(500)	status varchar(20)	doctor_id bigint
3	2026-04-06 13:20:20.77…	2026-04-08 13:00:00	Hello doctor! I need a…	PENDING	10



id bigint	created_at timestamp	date_time timestamp	notes varchar(500)	status varchar(20)	doctor_id bigint
3	2026-04-06 13:20:20.77…	2026-04-08 13:00:00	Hello doctor! I need a…	CONFIRMED	10

viii. After doctor's confirmation (Screenshot 2), the appointment's status has been changed to "CONFIRMED".

3. Implementation Summary

Main Feature: Appointment Booking and Doctor Availability Scheduling

The MedBuddy appointment module provides an end-to-end scheduling flow for patients and doctors. Patients can search for doctors, view available time slots, and book appointments, while doctors can define their availability, maintain recurring weekly schedules, add date-specific exceptions, and manage appointment status. The workflow is centered on authenticated users and is implemented through the appointment, availability, and schedule services in the backend, with the frontend consuming the corresponding REST endpoints.

Inputs and Validations

The module accepts structured JSON requests for availability management, slot creation, appointment booking, and appointment status updates. Appointment booking requires a doctor ID and slot ID, and notes are limited to 500 characters. Availability inputs require a date and, when applicable, valid start and end times; the system rejects past dates and enforces that start time must be earlier than end time. Additional service-level checks prevent patients from booking as doctors, prevent doctors from modifying slots they do not own, and restrict patients to canceling appointments only.

How the Feature Works

Doctors first define their schedule through a weekly template or by creating a date-based availability exception. When availability is saved, the backend generates or synchronizes 30-minute appointment slots based on the available time range, leaving a cutoff period before the end of clinic hours. Patients then view available slots for a doctor and book one that is still marked as available and lies in the future. Once booked, the slot is marked as booked and a corresponding appointment record is created. If an appointment is canceled, the slot becomes available again unless business rules prevent the change. This approach ensures that schedule data, slot generation, and appointment records remain consistent.

API Endpoints Used

The feature uses a mix of authentication, profile, doctor search, availability, slot, and appointment endpoints. The main endpoints are:

- POST /api/auth/register
- POST /api/auth/login
- GET /api/users/me
- GET /api/users/doctors
- GET /api/specializations
- GET /api/availability/doctor/{doctorId}
- GET /api/availability/doctor/{doctorId}/date/{date}
- POST /api/availability
- DELETE /api/availability/{date}
- POST /api/doctor/schedule/template
- GET /api/doctor/schedule/template
- POST /api/doctor/schedule/exception
- DELETE /api/doctor/schedule/exception/{date}

- POST /api/appointment-slots
- GET /api/appointment-slots/by-availability/{doctorAvailabilityId}
- GET /api/appointment-slots/by-doctor/{doctorId}?slotDate=...
- POST /api/appointments
- GET /api/appointments
- GET /api/appointments/{id}
- PATCH /api/appointments/{id}/status

Database Tables Involved

The main tables involved are users, patients, doctors, specializations, doctor_specializations, doctor_availability, doctor_schedule_templates, appointment_slots, and appointments. doctor_availability stores date-based availability windows, doctor_schedule_templates stores recurring weekly schedules, appointment_slots stores concrete bookable time slots, and appointments stores the actual booked consultations.

Why Some Deviations Were Necessary

The original SDD modeled appointments as a direct booking against a doctor's availability — patient picks a date, system checks availability, appointment is saved. This proved insufficient during implementation for the following reasons.

1. **appointment_slots table (new)** The SDD had no concept of discrete time slots. Without them, double-booking cannot be prevented at the data layer and no real availability calendar can be shown to patients. This table represents pre-generated bookable windows per doctor per day, each with an independent status that updates when a booking is made.
2. **doctor_availability.id column (added to existing table)** The original table had no primary key, making it unusable as a Hibernate JPA entity. An id column was added to enable repository operations and to serve as a foreign key target for appointment_slots.
3. **doctor_schedule_templates table (new)** The SDD assumed doctors enter availability manually per date. In practice, doctors follow a recurring weekly schedule. This table stores that weekly pattern, which a background job (SlotGenerationJob) reads to auto-generate slots for upcoming days.
4. **appointments.slot_id column (added to existing table)** Without a link back to the specific slot a booking consumed, the slot status could never be updated and queue position logic had no data to work from. This foreign key closes that gap.

Some deviations from the original SDD were necessary to support the implemented booking flow and to keep the schema consistent with the actual runtime behavior. The biggest change is that appointments are now slot-based rather than only date-and-time based, so the backend can prevent double-booking and safely release a slot when an appointment is canceled. This is why appointment_slots and the slot_id foreign key were added. The doctor_availability table also needed a surrogate id, because availability had to be referenced from generated slots even though the business rule still depends on the unique pair of doctor_id and available_date. In addition, doctor_schedule_templates was introduced to support recurring weekly schedules, which is more flexible than storing only isolated availability rows. Overall, the schema was expanded and normalized to match the implemented logic in AppointmentService.java, DoctorAvailabilityService.java, and V20260406_01__appointment_module_schema.sql.